

Marwadi University

BCA Semester - 3

05BC3301 – Database Management System – 2 (DBMS-2)

Unit – 4: Procedures, Functions and Packages



What is Procedure ?

- A ***stored procedure*** is a group of SQL and PL/SQL commands that execute certain task.
- A ***procedure*** is a group of PL/SQL statements that you can call by name.
- A ***procedure*** is a module performing one or more actions , it does not need to return any values
- The user must ***call a procedure*** either from a program or manually.
- ***Several users or applications*** can use a procedure.

Syntax for Procedure

```
CREATE [OR REPLACE] PROCEDURE <PROCEDURE_NAME>  
[Parameter1 {IN, OUT, IN OUT}, [Parameter2 {IN, OUT, IN  
OUT},...]]  
{IS/AS}  
    [CONSTANT / VARIABLE Declaration;]  
BEGIN  
    Executable statements;  
[EXCEPTION  
    Exception handling statements;]  
END;
```

Syntax for Procedure

- ***Procedure-name*** specifies the name of the procedure.
- ***[OR REPLACE]*** option allows modifying an existing procedure. It is optional.
- ***IN*** represents that value will be passed from outside and ***OUT*** represents that this parameter will be used to return a value outside of the procedure.
- ***procedure-body*** contains the executable part.
- The ***AS*** keyword is used instead of the ***IS*** keyword for creating a standalone procedure.

Procedure Parameters

- **IN** - This parameter passes a value into the program. This value is read only type of value. It cannot be change. It is the default mode.
- **OUT** - The OUT parameter passes a value back from the program. This value is write only value (printable value).
- **IN OUT** - This parameter passes the value into the program and returns the value from the program. This value is read and then it is written.

Procedure Parameters

■ Comparison between the procedure parameters:

IN	OUT	IN OUT
Default mode	Must be specified	Must be specified
Value is passed into subprogram	Returned to calling environment	Passed into subprogram; returned to calling environment
Formal parameter acts as a constant	Uninitialized variable	Initialized variable
Actual parameter can be a literal, expression, constant, or initialized variable	Must be a variable	Must be a variable
Can be assigned a default value	Cannot be assigned a default value	Cannot be assigned a default value

HOW TO EXECUTE OR RUN A PROCEDURE ?

SQL > SELECT * FROM <Table-name>;

SQL > @ <Procedure-file name>.SQL;

Procedure is successfully created...

OR

Procedure is created with compilation error...

SQL > SHOW ERRORS;

OR

SQL > SELECT * FROM USER_ERRORS;

SQL > EXECUTE <Procedure-name> (Parameter Value);

SQL > SELECT * FROM <Table-name>;

NOTE: - If your user doesn't have EXECUTE permission then write the following command.

SQL>GRANT EXECUTE ON <PROCEDURE-NAME> TO <USER-NAME>;

Deleting a Procedure

- **SYNTAX FOR DELETING A PROCEDURE:**

`DROP PROCEDURE <PROCEDURE_NAME>;`

- **EXAMPLE OF DELETING A PROCEDURE:**

`DROP PROCEDURE PROC1;`

Examples

Example 1:

-- The following example creates a simple procedure that displays the string 'Hello World!' on the screen when executed. (pe1.sql)

```
CREATE OR REPLACE PROCEDURE greetings AS  
  
BEGIN  
    dbms_output.put_line('Hello World!');  
END;  
/
```

```
SQL>@pe1.sql;
```

```
SQL>execute greetings;
```

Examples

Example 2:

-- Write a simple procedure without any parameter that updates the values in the EMP table. (pe2.SQL)

```
CREATE OR REPLACE PROCEDURE RAISE_SAL
IS
BEGIN
    UPDATE EMP SET SAL = SAL+SAL*0.10;
END;
/
```

```
SQL>@pe2.SQL;
```

```
SQL>EXECUTE RAISE_SAL;
```

Examples

Example 3:

-- Write a simple procedure that increases by the salary of employees for the given department no by percentage inputted by the user using IN parameter. (pe3.SQL)

```
CREATE OR REPLACE PROCEDURE RAISE_SAL2
(VDEPT IN EMP2.DEPTNO%TYPE, VPER IN NUMBER)
IS
BEGIN
    UPDATE EMP2 SET SAL=SAL+SAL*(VPER/100) WHERE
    DEPTNO=VDEPT;
END;
/
SQL>@pe3.SQL;
SQL>EXECUTE RAISE_SAL2 (10,20);
```

Examples

Example 4:

-- Write a procedure that search's whether the given employee number is present or not in the table. (pe4.SQL)

```
SET SERVEROUTPUT ON
```

```
CREATE OR REPLACE PROCEDURE SEARCH_EMP  
(VEMPNO IN NUMBER,VNAME OUT VARCHAR2)
```

```
IS
```

```
BEGIN
```

```
    SELECT ENAME INTO VNAME FROM EMP WHERE EID=VEMPNO;
```

```
EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE (TO_CHAR(VEMPNO) || ' NOT FOUND.');
```

```
END;
```

```
/
```

Examples

Example 4:

-- Write a procedure that search's whether the given employee number is present or not in the table.
(pe4.SQL) (Cont...)

```
SQL>@pe4.SQL;
```

```
SQL>VARIABLE EMP_NAME VARCHAR2(20);
```

```
SQL>EXECUTE SEARCH_EMP(100,:EMP_NAME);
```

```
SQL>PRINT EMP_NAME;
```

Examples

Example 5:

-- Write a PL/SQL block to call the SEARCH_EMP procedure. (pl5.sql)

SET SERVEROUTPUT ON

DECLARE

VEMPNO EMP.EMPNO%TYPE;

M_NAME EMP.ENAME%TYPE;

BEGIN

SEARCH_EMP (&VEMPNO, M_NAME);

DBMS_OUTPUT.PUT_LINE ('THE EMPLOYEE IS- ' || M_NAME || ' FOR
GIVEN EMPLOYEE NO');

EXCEPTION

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE (TO_CHAR (VEMPNO) || 'NOT FOUND');

END;

/

Examples

Example 6:

-- Write a procedure that shows the use of INOUT parameter.

(pl6.sql)

```
CREATE OR REPLACE PROCEDURE PS (no1 IN OUT NUMBER)
```

```
IS
```

```
BEGIN
```

```
    no1:=no1*no1;
```

```
END;
```

```
/
```

```
SQL > VARIABLE V NUMBER;
```

```
SQL > EXEC :V:=2;
```

```
SQL> EXECUTE PS(:V);
```

```
SQL> PRINT V;
```

What is User Defined Function ?

- **Functions** are also collection of SQL and PL/SQL code, which can return a value to the caller or to caller PL/SQL block.
- A **function** is similar to a **stored procedure**. The main **difference** between them is that function **returns a single value** and a stored procedure **doesn't return a value**.
- **Functions** can accept one, many, or no parameters, but a function must have a **return clause** in the executable section of the function.

Syntax for Function

```
CREATE [OR REPLACE] FUNCTION <FUNCTION_NAME>
    [Parameter1 {IN}, [Parameter2 {IN},...]]
RETURN DataType
{IS/AS}
    [CONSTANT / VARIABLE Declaration;]
BEGIN
    Executable statements;
[EXCEPTION
    Exception handling statements;]
RETURN Return_value;
END;
```

Syntax for Function

- ***function-name*** specifies the name of the function.
- ***[OR REPLACE]*** option allows modifying an existing function.
- The function must contain a ***return*** statement.
- ***RETURN*** clause specifies that data type you are going to return from the function.
- ***function-body*** contains the executable part.

HOW TO EXECUTE OR RUN A FUNCTION ?

- SQL > @ <function-filename>.SQL;

- SQL > SHOW ERRORS;

OR

- SQL > SELECT * FROM USER_ERRORS;

- SQL > VARIABLE <VARIABLE_NAME> datatype;

- SQL> EXECUTE :<VARIABLE_NAME>: =<function-name>(Parameter Value);

- SQL > PRINT <VARIABLE_NAME>;

OR

Write a PL/SQL block that calls the function

NOTE: - If your user doesn't have EXECUTE permission then write the following command.

SQL> GRANT EXECUTE ON <FUNCTION-NAME> TO <USER-NAME>;

Examples

Example 7:

-- Write a function that returns the total number of EMPLOYEES in the emp table. (f1.sql):

```
CREATE OR REPLACE FUNCTION total_emp  
RETURN number  
IS  
    total number(2) := 0;  
BEGIN  
    SELECT count(*) into total FROM emp;  
    RETURN total;  
END;  
/
```

How to call a function from command line ?

Example 7:

```
SQL> @ F:/PLSQL/f1.sql;
```

```
SQL> var mno number;
```

```
SQL> execute :mno:=total_emp;
```

PL/SQL procedure successfully completed.

```
SQL> print mno;
```

MNO

14

How to call a function from separate pl block?

Example 8:

-- Write a pl block that calls a total_emp function of example 7 (cf1.sql).

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    VNO          NUMBER(5);
```

```
BEGIN
```

```
    VNO:=total_emp;
```

```
    DBMS_OUTPUT.PUT_LINE('Total no of records in emp table are:-'  
                          '||VNO);
```

```
END;
```

```
/
```

Examples

Example 9:

-- Write a function that computes and returns the maximum of two values (f2.sql).

```
CREATE OR REPLACE FUNCTION FindMax(x IN number, y IN number)
```

```
RETURN number
```

```
IS
```

```
    z number;
```

```
BEGIN
```

```
    IF x > y THEN
```

```
        z:= x;
```

```
    ELSE
```

```
        z:= y;
```

```
    END IF;
```

```
RETURN z;
```

```
END;
```

```
/
```

Examples

Example 9:

-- Write a function that computes and returns the maximum of two values (f2.sql). (Cont...)

```
SQL> var ans number;
```

```
SQL> execute :ans:=findMax(15,5);
```

```
SQL> print ans
```

Examples

Example 10:

-- Write a function that returns the square of the given number (f3.sql)

```
CREATE OR REPLACE FUNCTION FSQUARE (NO IN NUMBER)
```

```
RETURN NUMBER
```

```
IS
```

```
BEGIN
```

```
    RETURN NO*NO;
```

```
END;
```

```
/
```

```
SQL> var res number;
```

```
SQL> execute :res:=FSQUARE (15);
```

```
SQL> print res
```

Difference between a Procedure and Function

<u>Stored Procedure</u>	<u>User-defined functions</u>
Can Return zero or n values.	Can return only one value which is mandatory.
Can have input/output parameters for it.	Can have only input parameters.
Allows select as well as DML statement in it.	Allows only select statement in it.
Procedures cannot be called from function.	Functions can be called from a stored procedure.
Exception can be handled by try-catch block	try-catch block cannot be used in a function.
We can go for transaction management in procedures.	We can't go for transaction management in functions.
Can not be utilized in a select statement	Can be embedded in a select statement.
Stored procedures cannot be used in the SQL statements.	UDF can be used in the SQL statements anywhere in the WHERE/HAVING/SELECT section.